Comprehensive Technical Progress Report

Empirical Evaluation of Multi-Agent Systems

Semester Project Documentation

June 13, 2026

Abstract

This report documents the complete empirical workflow, experimental bottlenecks, and quantitative results obtained during the evaluation of the OpenHands CodeAct architecture. It details the execution of HumanEval tasks (#1 through #10) and the transition to repository-scale SWE-bench tasks. The report highlights critical difficulties faced, prompt engineering strategies, and the "Self-Healing" phenomenon observed in massive LLMs compared to local framework collapse.

1 Complete Experimental Workflow

The research followed a strict empirical workflow to test multi-agent capabilities under varying cognitive loads:

1. **Environment Deployment:** OpenHands was deployed via Docker, mapped to a local Ubuntu/WSL filesystem.
2. **Role Configuration:** We established two primary test setups:
   - **Setup A (Coder Only):** The agent is constrained solely to code generation.
   - **Setup B (Pair Programming):** The agent acts as both Coder and Tester, required to write logic, generate `unittest` scripts, and execute them in the bash terminal.
3. **Model Selection:** After local models failed due to hardware limits, we routed the framework to the `cerebras/gpt-oss-120b` Cloud API.
4. **Sequential Execution:** We executed HumanEval tasks #1 through #10, followed by a full SWE-bench repository evaluation.

2 Detailed Difficulties Faced

Throughout the execution, several critical framework and infrastructure bottlenecks were documented:

- **Difficulty 1: Hardware Exhaustion & 7B Collapse.** Initial tests with local Ollama models (7B) caused severe Docker timeouts. Smaller models completely failed to follow the XML tool-calling format, hallucinating tags and crashing the system.

- **Difficulty 2: API Schema Brittleness.** The OpenHands `str_replace_editor` tool strictly requires a `security_risk` parameter. Almost all models naturally omitted this parameter, resulting in framework exceptions.
- **Difficulty 3: File Conflicts.** During HumanEval #9, the model attempted to create a file that already existed, throwing a path error.
- **Difficulty 4: Token Rate Limits (TPM).** When executing SWE-bench repository tests, the terminal outputted massive `pip install` progress bars and verbose `pytest` stacks. This flooded the 120B model's context window, causing fatal "Tokens Per Minute Limit Exceeded" API crashes.
- **Difficulty 5: Cross-Contamination of Tests.** Global `pytest` commands accidentally triggered leftover HumanEval test files, confusing the agent with unrelated `ModuleNotFoundError` outputs.

3 Tasks Given to OpenHands (The Prompts)

To overcome the aforementioned difficulties, highly specific prompt engineering was required.

3.1 HumanEval Prompts

Setup A (Coder Only) Template:
"You are a Coder agent. Your ONLY job is to write code. Do not create a task list, do not write test scripts, and do not execute terminal commands. Please use your code editor tool to write a Python file named humaneval_[ID].py in the /workspace directory that solves this exact problem: [TASK DETAILS]"

Setup B (Pair Programming) Template:
"You are a Pair Programming team (Coder + Tester). Do not create a task list. Please use your code editor tool to write a Python file named humaneval_[ID].py in the /workspace directory that solves the following problem. After saving it, act as the Tester: use your code editor tool to write a separate script called test_humaneval_[ID].py to test the logic, and then execute it in the bash terminal to verify your code passes."

3.2 SWE-bench Targeted Prompt

To bypass Token Rate Limits (Difficulty 4) and Cross-Contamination (Difficulty 5), the following constrained prompt was used:
"You are a SWE-bench evaluation agent. I have already installed all required dependencies. Do NOT run any pip install commands. Your task is to resolve the failing tests in the codebase. You MUST explicitly target the correct test files. Run exactly: `pytest test_calculator.py test_app.py -q`. Read the small chunk of the failing test, find the bug, and fix it using `str_replace_editor` (including the `security_risk` parameter)."

4 Complete HumanEval Execution Results (#1–#10)

The following matrix documents the complete sequence of tests. Tasks #1 and #2 demonstrate initial failures, while Tasks #3 through #10 highlight the **Self-Healing** phenomenon where the 120B model autonomously recovered from framework errors.

Task	Benchmark	Config.	Model	Result	Exception Caught	Resolution Strategy
#1	HumanEval	Setup A	7B (Local)	**FAIL**	100% Core Collapse	Infrastructure Timeout & Schema Hallucination
#2	HumanEval	Setup A	120B (Cloud)	**FAIL**	Missing `security_risk`	Did not recover / Re-prompt required
#3	HumanEval	Setup B	120B (Cloud)	**PASS**	None	Executed perfectly on first attempt
#4	HumanEval	Setup A	120B (Cloud)	**PASS**	None	Executed perfectly on first attempt
#5	HumanEval	Setup A	120B (Cloud)	**PASS**	None	Executed perfectly within same chat context
#6	HumanEval	Setup B	120B (Cloud)	**PASS**	Missing `security_risk`	**Self-Healed** (Auto-re-called tool with parameter)
#7	HumanEval	Setup A	120B (Cloud)	**PASS**	Missing `security_risk`	**Self-Healed** (Auto-re-called tool with parameter)
#8	HumanEval	Setup B	120B (Cloud)	**PASS**	Missing `security_risk`	**Self-Healed** (Wrote logic + tests successfully)
#9	HumanEval	Setup A	120B (Cloud)	**PASS**	File Conflict Error	**Autonomous Pivot** (Switched to `view` to verify)
#10	HumanEval	Setup B	120B (Cloud)	**PASS**	Missing `security_risk`	**Self-Healed** (Wrote full `unittest` suite autonomously)

Table 1: Complete HumanEval Execution Matrix tracking 1 to 10.

5 SWE-bench Execution Results

Following the HumanEval phase, the agent was evaluated on a multi-file repository (`calculator.py` and `app.py`).

- **Initial Attempts:** Failed due to Token Rate Limits from verbose `pip` downloads and global `pytest` errors.
- **Final Execution:** After applying strict environmental constraints (pre-installing dependencies and targeting tests via `pytest -q`), the agent successfully ingested the repository structure, read the targeted files, and verified the logic.
- **Final Output:** `7 passed in 0.26s` (100% Success Rate under constrained tokens).

6 Conclusion

This testing cycle proves two major empirical findings. First, massive parameter models possess an emergent **Self-Healing** capability, allowing them to autonomously correct XML framework errors that cause total crashes in local 7B models. Second, while highly capable of complex logic, multi-agent frameworks are severely bottlenecked by context window exhaustion from standard CI/CD terminal noise, requiring strict Prompt Engineering to truncate logs (`head -n 20`) and enforce quiet execution.